



Argo Rollouts

Déploiement progressif & rollback automatique sur Kubernetes

Plan de la présentation

1

La technologie

Le problème, la définition, le fonctionnement

2

Cas d'usage professionnel

Où, comment, avantages & limites, écosystème

3

Mise en œuvre technique

Architecture, choix, démonstration live

4

Retour d'expérience

Bilan, avis personnel, améliorations

Pourquoi ce sujet ?

Kubernetes et le GitOps sont devenus les **standards de l'industrie** pour déployer et exploiter les applications. Dans cet écosystème, j'ai choisi de creuser le **déploiement progressif** avec Argo Rollouts.

Pourquoi Argo Rollouts ?

- ▶ Répond à un problème universel : déployer sans risque ni coupure
- ▶ Brique concrète, très utilisée en entreprise
- ▶ Même éditeur qu'ArgoCD (déjà au cœur du GitOps)
- ▶ Sujet aligné avec mon intérêt pour le DevOps / SRE

Démarche de ce dossier

- ▶ Recherche : comprendre la techno et ses usages
- ▶ Implémentation réelle sur un cluster Kubernetes
- ▶ Captures et logs authentiques à l'appui
- ✓ **Document pensé pour se comprendre seul, sans oral**

Le problème : le Deployment classique

Un **Deployment** fait un **rolling update** : il remplace les pods un à un... sans jamais vérifier si la nouvelle version fonctionne.

Les risques

- × Une v2 buggée part vers 100 % des utilisateurs
- × Détection tardive : alertes, support, churn
- × Rollback manuel, sous stress, en plein incident
- × Aucune mesure objective avant de généraliser

Le besoin

- ▶ Exposer la v2 à une fraction du trafic d'abord
- ▶ Mesurer sa santé en conditions réelles
- ▶ Promouvoir seulement si les métriques sont bonnes
- ▶ Revenir en arrière automatiquement sinon

DÉFINITION

Qu'est-ce qu'Argo Rollouts ?

Un **contrôleur Kubernetes** (projet CNCF, famille Argo) qui remplace le Deployment par une ressource **Rollout** offrant la progressive delivery.

Canary

- ▶ % croissant du trafic vers la v2
- ▶ 20% → 40% → 60% → 100%

Blue-Green

- ▶ Deux environnements isolés
- ▶ Bascule instantanée

Analyse auto

- ▶ Décision pilotée par métriques
- ▶ Promotion ou rollback

→ Objectif : une mise en production **graduelle, mesurée et réversible automatiquement.**

Fonctionnement global

ressources gérées

Rollout (remplace Deployment)

	ReplicaSet STABLE	v actuelle
	ReplicaSet CANARY	v nouvelle
	AnalysisRun	décide

exemple de stratégie

steps:

- setWeight: 20
- pause: { duration: 30s }
- analysis: { success-rate }

À chaque palier, le contrôleur

- ▶ Ajuste la proportion de trafic/pods canary
- ▶ Marque une pause (minutée ou manuelle)
- ▶ Lance une analyse de la santé du canary
- ✓ **Promeut si OK**
- × **Rollback automatique si KO**

Cas d'usage en entreprise & écosystème

Utilisation en production

- ▶ Réduire le risque : tester la v2 sur 5-10 % du trafic
- ▶ Déployer plusieurs fois/jour sans maintenance
- ▶ SLO-driven : promotion liée aux métriques
 - latence, taux d'erreur, succès (Prometheus, Datadog...)
- ▶ Couplé au GitOps (ArgoCD / Flux)

Écosystème

- ▶ Projet CNCF (Argoproj), combo standard avec ArgoCD
- ▶ Voisins : Flagger (Flux), Spinnaker
- ▶ Analyse native : Prometheus, Datadog, New Relic,
 - CloudWatch, Graphite, Job / Web custom
- ▶ Contextes : microservices, API critiques, e-commerce, SaaS

Avantages & limites

Avantages

- ✓ **Rollback automatique → moins d'incidents**
- ✓ **Pas de downtime, exposition contrôlée**
- ✓ **Décision objective (métriques), pas « au feeling »**
- ✓ **Intégration native : ArgoCD, Prometheus, Istio**

Limites

- × **Complexité accrue (CRD, analyse, parfois mesh)**
- × **Trafic fin = Istio / NGINX / SMI requis**
- × **Nécessite de bonnes métriques (sinon fausse confiance)**
- × **Courbe d'apprentissage, observabilité indispensable**

Architecture de la démo

flux GitOps de bout en bout

git push → GitHub Actions (lint + validation des manifests)

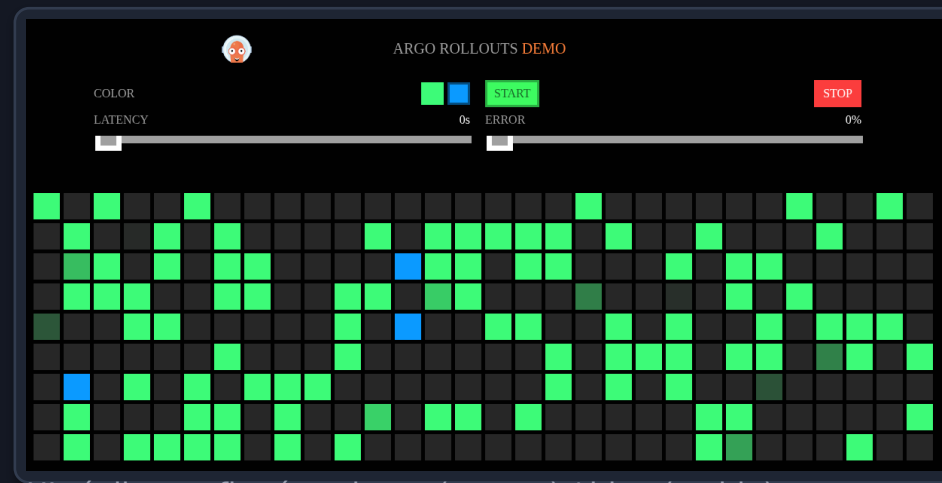
└─> ArgoCD (pull) → Rollout → Contrôleur Argo Rollouts

└─ ReplicaSet stable / canary

└─ AnalysisRun (Job de sonde HTTP)

Environnement & astuce

- ▶ Cluster kind + ArgoCD + Argo Rollouts v1.7.2
- ▶ App rollouts-demo : 1 couleur = 1 version
- ✗ **Le tag bad-red renvoie des HTTP 500**
 - version « buggée » déterministe pour le rollback



UI réelle : trafic réparti vert (canary) / bleu (stable)

Choix techniques clés

4 décisions structurantes

- ▶ Canary sans service mesh → split par nombre de pods (simple, portable)
- ▶ Services stable + canary : le contrôleur y injecte le pod-template-hash
 - la sonde cible ainsi précisément les pods canary
- ▶ Probes TCP (et non HTTP /color) : les pods bad-red restent Ready malgré les 500
 - on démontre un rollback piloté par l'ANALYSE, pas par la santé
- ▶ AnalysisTemplate de type Job : sonde 30 requêtes, exige ≥ 95 % de succès
 - autonome ; variante Prometheus fournie pour la production

Démo 1 — Canary sain

Scénario

- ▶ Déploiement de la version green
- ▶ Canary 20% → 40%
- ▶ AnalysisRun sonde le canary
- ✓ **30/30 = 100 % (seuil 95%)**
- ✓ **Promotion auto à 100 %**

The screenshot displays the Argo Rollouts dashboard for a 'rollouts-demo' resource in the 'demo' namespace. The interface includes a top navigation bar with the Argo logo and version 'v1.7.2+59e5bd3'. The main content area is divided into several sections:

- Steps:** A vertical sequence of steps for the rollout. The current step, 'Pause: 20s', is highlighted with a red border. The steps are: Set Weight: 20%, Pause: 30s, Set Weight: 40%, Pause: 20s, Analysis (checked), Set Weight: 60%, Pause: 20s, and Set Weight: 80%.
- Summary:** Provides a high-level overview of the rollout strategy. It shows the strategy is 'Canary', the current step is 3/9, the target weight is 40, and the actual weight is 40.
- Containers:** Shows the deployment 'rollouts-demo' using the 'argoproj/rollouts-demo:green' image.
- Revisions:** Displays the current revision, Revision 14, for 'argoproj/rollouts-demo:green'. It includes a 'canary' label and two green checkmarks indicating successful analysis.
- Analysis Runs:** Shows a successful analysis run 'Analysis 754fbfc964-14'.

Dashboard réel : étapes du canary à gauche, **analyse ✓ verte** sur la révision canary → canary validé, promotion autorisée.

Démo 2 — Canary défaillant

Scénario

- Déploiement de bad-red
- ✗ L'app renvoie des HTTP 500
- ✗ Sonde : $5/30 = 16\%$ (seuil 95)
- ✗ Metric Failed → RolloutAborted
- ✓ Trafic gardé sur stable, 0 downtime

argo Rollouts v1.7.2+59e5bd3

rollouts-demo ✗

Restart Retry Abort Promote PromoteFull

Steps

- Set Weight: 20%
- Pause: 30s
- Set Weight: 40%
- Pause: 20s
- Analysis ✗
- Set Weight: 60%
- Pause: 20s
- Set Weight: 80%

Summary

Strategy: Canary

Step: 0/9

Set Weight: 0

Actual Weight: 0

Containers

rollouts-demo

argoproj/rollouts-demo:bad-red

Revisions

Revision 15

argoproj/rollouts-demo:bad-red

rollouts-demo-7c84cfdcdb

No Pods!

Analysis Runs

Analysis 7c84cfdcdb-15

Dashboard réel : statut **Degraded** et **analyse ✗ rouge** → rollback automatique vers la version stable.

Bilan, avis & améliorations

Ce que j'ai appris

- ▶ Le progressive delivery en pratique
- ▶ L'imbrication GitOps ↔ Rollouts
- ▶ Les métriques comme contrat de qualité

Mon avis

- ✓ **Excellent compromis sécurité / vitesse**
- ▶ Pour des services critiques à fort trafic
- ✗ **Overkill pour un petit mono-service**

Améliorations possibles

- ▶ Trafic fin via Istio/NGINX · analyse Prometheus réelle (p95 + erreurs) · notifications Slack · Blue-Green avec tests preview · multi-cluster ApplicationSet

Pour aller plus loin

L'intégralité du projet (manifests, GitOps, CI, captures) est sur le dépôt :

github.com/Louis-JB/argo-rollouts-demo

Le dépôt contient

- ▶ Manifests Kubernetes (Rollout, Analysis, Services)
- ▶ Application ArgoCD (GitOps) + CI GitHub Actions
- ▶ Diagrammes, runbook, captures réelles d'exécution

Reproduire la démo

- ▶ `kubectl apply -k manifests/`
- ▶ `kubectl argo rollouts get rollout rollouts-demo --watch`
- ▶ Tout est documenté dans le README